

HTB ACADEMY

JavaScript Deobfuscation

HTB Academy · Module 41 · Malware Analysis / Blue Team Track · Study Notes

Condensed study notes for all 10 sections of the module, with a front-loaded command index. Flags are partially masked (first/middle/last character shown) in line with HTB's sharing guidance.

Module	JavaScript Deobfuscation (#41)
Completion	100% — all 10 sections
Analyst	Chayan Panchal · @tron874
Format	Quick-reference index first, then topic-by-topic notes

Command & Tool Index — Read This First

Every command and tool used in this module, grouped by purpose.



Source inspection

CTRL+U	View raw HTML source
CTRL+SHIFT+Z	Open Firefox Debugger (DevTools)
{ } button (DevTools)	Pretty-print / beautify minified JS



Deobfuscation tools (online)

prettier.io/playground	Format and indent obfuscated JS
beautifier.io	Alternative JS/HTML/CSS beautifier
matthewfl.com/unPacker.html	Unpack Dean Edwards eval(p,a,c,k,e,d)
jsconsole.com	Run JS snippets, verify output
boxentriq.com/cipher-identifier	Guess an unknown encoding/cipher



Encoding / decoding + curl cheat-sheet

echo "TEXT" base64	# Base64 encode
echo "B64" base64 -d	# Base64 decode
echo "TEXT" xxd -p	# Hex encode
echo "HEX" xxd -p -r	# Hex decode
echo 'TEXT' tr 'A-Za-z' 'N-ZA-Mn-za-m'	# ROT13 (same cmd both ways)
curl http://TARGET/	# GET request
curl -s http://TARGET/	# silent mode
curl -X POST http://TARGET/serial.php	# POST, no body
curl -X POST URL -d "k=v"	# POST with form data
curl -X POST URL -H "Content-Type: application/json" -d '{"key":"val"}'	
curl -v http://TARGET/	# verbose — see headers

1 What Obfuscation Is & Why Attackers Use It

Obfuscation $\neq$ encryption. The code still runs identically — it's just transformed to be extremely hard for a human to read at a glance.

Use case	Why
Hide malware payloads	Conceal C2 URLs, download commands, dropper logic
Evade IDS/IPS	Signature detection can't match patterns inside obfuscated code
Protect IP	Slows down reverse engineering of proprietary logic
Client-side auth bypass	Hides API keys/tokens in JS — insecure but common

Four patterns to recognise on sight:

Pattern	Tell
Minification	One line, no whitespace — .min.js
Packing (Dean Edwards)	eval(function(p,a,c,k,e,d){...} signature
obfuscator.io	_0x-prefixed vars, Base64 string arrays
JSFuck	Only [] () ! + characters, very verbose

2 Deobfuscation Workflow & Result



secret.js — unpacked (flag masked)

```
'use strict';
function generateSerial() {
  var flag = 'HTB{n.....u.....!}';
  var xhr = new XMLHttpRequest();
  var url = '/serial.php';
  xhr.open('POST', url, true);
  xhr.send(null);
};
```

NOTE — Finding a flag/secret hard-coded in client-side JS is a real anti-pattern. Anything shipped to the browser should be treated as public — that's the lesson this exercise is really teaching.

3 Encoding & Decoding — Base64, Hex, ROT13

Type	What it is	How to spot it
Base64	Binary represented in 64 safe printable chars	Alphanumeric + / =, length divisible by 4
Hex	Each byte as 2 hex digits (base 16)	Only 0-9 a-f, always an even length
ROT13	Each letter shifted 13 positions	Looks like scrambled English — http -> uggc

```
Base64

# ENCODE
$ echo "https://www.hackthebox.eu/" | base64
aHR0cHM6Ly93d3cuaGFja3RoZWJveC5ldS8K

# DECODE (-d flag = decode)
$ echo "aHR0cHM6..." | base64 -d
https://www.hackthebox.eu/

# LAB EXAMPLE
$ echo "N2gxNV8xNV9hX3MzY3IzN19tMzU1NGcz" | base64 -d
7.....c.....3
```

NOTE — The = or == at the end is padding, not data. Base64 length must be divisible by 4, so = characters fill the gap. One = means 1 padding char, == means 2.

```
Hex

# ENCODE
$ echo "https://www.hackthebox.eu/" | xxd -p
68747470733a2f2f7777772e6861636b746865626f782e65752f0a

# DECODE (-p -r = plain hex, reverse)
$ echo "687474..." | xxd -p -r
https://www.hackthebox.eu/

# SKILLS ASSESSMENT EXAMPLE
$ echo "4.....3.....e" | xxd -p -r
A.....7.....n      <- decoded API key
```

```
ROT13

# ENCODE
$ echo 'https://www.hackthebox.eu/' | tr 'A-Za-z' 'N-ZA-Mn-za-m'
uggcf://jjj.unpxgurobk.rh/

# DECODE — exact same command, ROT13 x2 = original
$ echo 'uggcf://jjj.unpxgurobk.rh/' | tr 'A-Za-z' 'N-ZA-Mn-za-m'
https://www.hackthebox.eu/
```

TIP — If you're unsure which encoding you're looking at, paste the string into boxentriq.com/cipher-identifier — it will suggest the most likely type.

4 HTTP Requests with curl

Flag	Meaning
(none)	Basic GET request — fetch page HTML
-s	Silent mode — suppress progress output
-v	Verbose — show full request AND response headers
-X POST	Send a POST request (matches xhr.open("POST",...) in JS)
-d "k=v"	Attach POST form data; chain with & for multiple params
-H "header"	Set a custom header, e.g. Content-Type: application/json



Full lab sequence — Sections 8 & 9

```
# Step 1 — replicate the generateSerial() POST
$ curl -s http://SERVER_IP:PORT/serial.php -X POST
7.....c.....3 (masked)

# Step 2 — identify encoding -> Base64 -> decode
$ echo "..." | base64 -d
7.....c.....3

# Step 3 — send decoded value back to get the flag
$ curl -s http://SERVER_IP:PORT/serial.php \
      -X POST -d "serial=7.....c.....3"
HTB{j.....r.....l}
```

5 Skills Assessment — Step-by-Step Solution

Q1 — Find the JavaScript file

CTRL+U to view source, scan for script src= tags.

```
Terminal / Browser

$ curl -s http://TARGET/ | grep script
<script src="api.min.js"></script>
```

Answer api.min.js

Q2 — Run the obfuscated code

Paste api.min.js into jsconsole.com, press Enter — console.log prints a flag.

```
Terminal / Browser

jsconsole.com -> paste api.min.js -> Enter
> [output]
```

Answer HTB{j.....m.....y}

Q3 — Deobfuscate, extract the flag variable

UnPacker reveals the flag variable in plaintext.

```
Terminal / Browser

unpacker -> deobfuscate api.min.js
```

Answer HTB{n.....u.....!}

Q4 — Replicate the POST, get the secret key

POST to /keys.php with no body, replicated with curl.

```
Terminal / Browser

$ curl -s http://TARGET/keys.php -X POST
```

Answer 4.....3.....e

Q5 — Decode hex, get the final flag

Response is hex — decode with xxd, POST it back.



Terminal / Browser

```
$ echo "4.....3.....e" | xxd -p -r -> A.....7.....n  
$ curl -s ../keys.php -X POST -d "key=A.....7.....n"
```

Answer

HTB{r.....m.....B}

6 Bonus Reference — Beyond This Module

Not part of Module 41 itself, but this is the surrounding context a working malware/blue-team analyst keeps in their back pocket when triaging obfuscated JavaScript for real.

Other obfuscation families you'll run into outside this module

Family	Tell
AAEncode / JJEncode	Code written almost entirely in Japanese emoticon-style characters
Base62 / custom alphabets	Looks like Base64 but doesn't decode cleanly — check the charset
Control-flow flattening	A single while/switch loop drives all execution order — hard to trace statically
String concatenation obfuscation	Strings built char-by-char or via array index lookups at runtime

Static vs. dynamic analysis — know which one you're doing

Approach	What it means	Risk
Static analysis	Reading the code without running it — everything in this module	Safe, but obfuscation can defeat it
Dynamic analysis	Actually executing the code (e.g. in jsconsole.com) to observe behaviour	Only ever do this in an isolated/sandboxed environment

CAUTION — Running unknown JavaScript from a real malware sample — as opposed to a controlled HTB lab target — should only ever happen inside an isolated VM or sandbox with no network access to anything sensitive. jsconsole.com is fine for lab exercises; it is not a malware sandbox.

Where this connects to real detection engineering

CONNECTION — Everything decoded by hand here (Base64, hex, ROT13) is exactly what SIEM/EDR string- and entropy-based detections are built to flag automatically — a sudden spike in high-entropy strings inside a JS file, or an XHR to a newly-registered domain, is the kind of signal that turns this manual workflow into an automated alert.